

Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación

Unidad gestora: *Escuela de Ciencias Básicas Tecnología e Ingeniería ECBTI*

Programa: *Ingeniería de Sistemas*

Curso: Programación

Código: 213023

**Propuestas para el desarrollo de la Fase 3 del curso
Programación**

En el presente documento se presentan cinco (5) propuestas de proyecto que serán desarrolladas en el marco de la Fase 3. El propósito de cada proyecto es permitir la aplicación práctica de los conocimientos adquiridos durante el curso.

Cada estudiante deberá seleccionar un (1) único proyecto para su desarrollo. El proyecto debe implementarse utilizando el lenguaje de programación Python y la biblioteca estándar Tkinter. Además, todas las interfaces gráficas deben estar diseñadas en idioma inglés.

Ejercicio 1: Sistema de Vehículos Inteligentes

En una empresa dedicada al desarrollo de movilidad inteligente, se está diseñando un prototipo de software que permita simular distintos tipos de vehículos y su comportamiento según su tecnología, su aceleración y sus capacidades de frenado. El objetivo principal del sistema es crear una arquitectura de clases que sea extensible, fácil de mantener y que

permita añadir nuevos tipos de vehículos en el futuro sin reescribir grandes fragmentos de código.

Para lograr esto, se debe implementar una clase base llamada **Vehículo**, que contenga los atributos comunes para todos los vehículos como *marca*, *modelo* y *velocidad_actual*. Esta clase base también debe incluir métodos que representen las acciones generales:

- `acelerar()`
- `detener()`
- `obtener_informacion()`

El método `acelerar()` debe demostrarse por **sobrecarga** utilizando múltiples formas de ejecutarlo, por ejemplo:

- `acelerar()` sin parámetros (incremento base)
- `acelerar(turbo)` con un parámetro opcional para acelerar más rápido
- `acelerar(turbo, terreno)` para considerar las condiciones del terreno

Posteriormente, el estudiante debe crear al menos **tres clases hijas** que hereden de `Vehículo`:

1. **AutoElectrico**: acelera de forma progresiva y consume energía.
2. **Moto**: acelera rápidamente pero tiene menor estabilidad.
3. **Camion**: acelera lentamente y tiene mayor peso.

Cada una de estas clases debe **sobrescribir** (polimorfismo) los métodos `acelerar()` y `detener()` para reflejar el comportamiento específico del vehículo.

Finalmente, el estudiante debe crear una lista con múltiples objetos de distintas clases y simular una prueba donde todos los vehículos aceleren y se detengan usando un único ciclo que llame los mismos métodos. Esto debe demostrar polimorfismo, ya que cada vehículo responde de forma distinta aunque se utilice la misma llamada desde el código principal.

Ejercicio 2: sistema Avanzado de Figuras 3D

Una empresa dedicada a la animación digital requiere un módulo matemático especializado para calcular áreas superficiales y volúmenes de distintos objetos tridimensionales que serán renderizados en escenas de alta calidad. El sistema debe ser altamente reutilizable, permitir la introducción de nuevas figuras en futuras versiones y ser flexible para manejar cálculos con escalado dinámico.

Se debe crear una clase abstracta llamada **Figura3D**, que incluya:

- Método abstracto `calcular_volumen()`
- Método abstracto `calcular_area_superficial()`
- Método polimórfico `dibujar()` que cada figura utilizará para mostrar su representación textual

Luego se deben implementar las clases:

- **Cubo** (lado)
- **Esfera** (radio)
- **Cilindro** (radio y altura)

Cada figura debe sobrescribir sus métodos de cálculo y su método `dibujar()` con una descripción única.

Además, se debe incluir **sobrecarga** en el método `calcular_area_superficial()`, creando una versión que permita recibir un parámetro opcional llamado `factor_escalado`. Esto permitirá simular figuras más grandes o más pequeñas sin crear nuevas instancias.

La aplicación debe incluir:

- Una lista de figuras mezcladas.
- Un ciclo que invoque `dibujar()`, `calcular_area_superficial()` y `calcular_volumen()` sobre cada figura sin conocer su tipo.
- Simulación de escalado dinámico a través de la sobrecarga del método.

Ejercicio 3: Sistema de Nómina Empresarial

Una compañía multinacional quiere modernizar su sistema de nómina para centralizar el cálculo de salarios en un solo motor que pueda ser extendido sin modificar el código base. El sistema debe soportar múltiples tipos de empleados que se paguen bajo reglas diferentes.

Se debe crear la clase base **Empleado** con los atributos generales:

- nombre
- identificacion
- salario_base

La clase base debe incluir un método virtual `calcular_salario()` y un método `mostrar_informacion()`.

Luego se deben crear tres clases hijas con comportamientos específicos:

1. **EmpleadoTiempoCompleto:** recibe salario fijo más bonificaciones por desempeño.
2. **EmpleadoPorHoras:** recibe salario base calculado por número de horas trabajadas.
3. **EmpleadoComision:** recibe salario base más un porcentaje por ventas.

Además, se debe simular **herencia múltiple** combinando empleados con una clase extra llamada **Bonificable**, que contenga:

- agregar_bonificacion(monto)
- obtener_bonificaciones()

La clase hija **EmpleadoTiempoCompletoBonificado** debe mostrar cómo un empleado puede heredar comportamiento de dos clases (o simularlo mediante mixins o composición).

El estudiante debe también sobrecargar el método `mostrar_informacion()` de forma que pueda:

- Mostrar solo los datos básicos
- Mostrar salario total
- Mostrar salario + bonificaciones + detalles según parámetros opcionales

Finalmente, se debe crear un sistema que gestione una lista de empleados y calcule la nómina mensual mediante polimorfismo.

Ejercicio 4: Sistema de Tienda Virtual

Una plataforma de comercio electrónico quiere modelar un sistema que permita gestionar diferentes tipos de productos y aplicar ofertas especiales según la categoría. El objetivo del ejercicio es diseñar una jerarquía de clases que permita que los productos compartan lógica común, pero que cada uno pueda extender sus funcionalidades según sus necesidades particulares.

Crear una clase base llamada **Producto**, con:

- nombre
- precio
- stock
- Método polimórfico `descripcion()`
- Método `calcular_descuento(porcentaje)`
- Método `calcular_precio_final()`

Luego crear las clases hijas:

- **ProductoElectronico**
- **ProductoAlimento**
- **ProductoRopa**

Cada tipo de producto debe sobrescribir:

- `descripcion()`
- `calcular_descuento()` según reglas propias
 - Por ejemplo, ropa con descuentos acumulables
 - Alimentos con descuento máximo permitido

- Electrónicos con depreciación

Además, se debe **sobrecargar** métodos como:

- `calcular_descuento(descuento, cupon)`
- `calcular_descuento(descuento, cupon, cliente_premium)`

Por último, se debe diseñar una clase **CarritoDeCompras** que almacene productos de distintos tipos y recorra la lista utilizando polimorfismo para obtener descripciones y calcular el monto total de la compra.

Ejercicio 5: Sistema de Control de Dispositivos Inteligentes

Una empresa desarrolladora de tecnología para hogares inteligentes desea un sistema que permita controlar distintos dispositivos desde un panel central. El software debe estar diseñado para que en el futuro puedan añadirse nuevos dispositivos sin reescribir código existente.

Crear la clase base **Dispositivo**, que incluya:

- nombre
- estado (encendido/apagado)
- Método `encender()`
- Método `apagar()`
- Método polimórfico `status()`

Luego se deben crear las clases concretas:

- **BombillaInteligente**
- **CortinaInteligente**
- **TermostatoInteligente**

Cada clase debe sobrescribir:

- encender()
- apagar()
- status()

Además, se debe implementar la **sobrecarga** del método configurar() en cada dispositivo según distintos escenarios, por ejemplo:

- configurar(modos)
- configurar(modos, intensidad)
- configurar(modos, intensidad, hora_programada)

Finalmente, crear la clase **ControlCentral**, que almacene múltiples dispositivos y permita:

- Encender todos los dispositivos (polimorfismo)
- Apagar todos
- Consultar estado general
- Configurar dispositivos individualmente
- Recorrer una lista donde cada dispositivo responde de forma distinta aunque se invoquen los mismos métodos